Classical ML to NNs

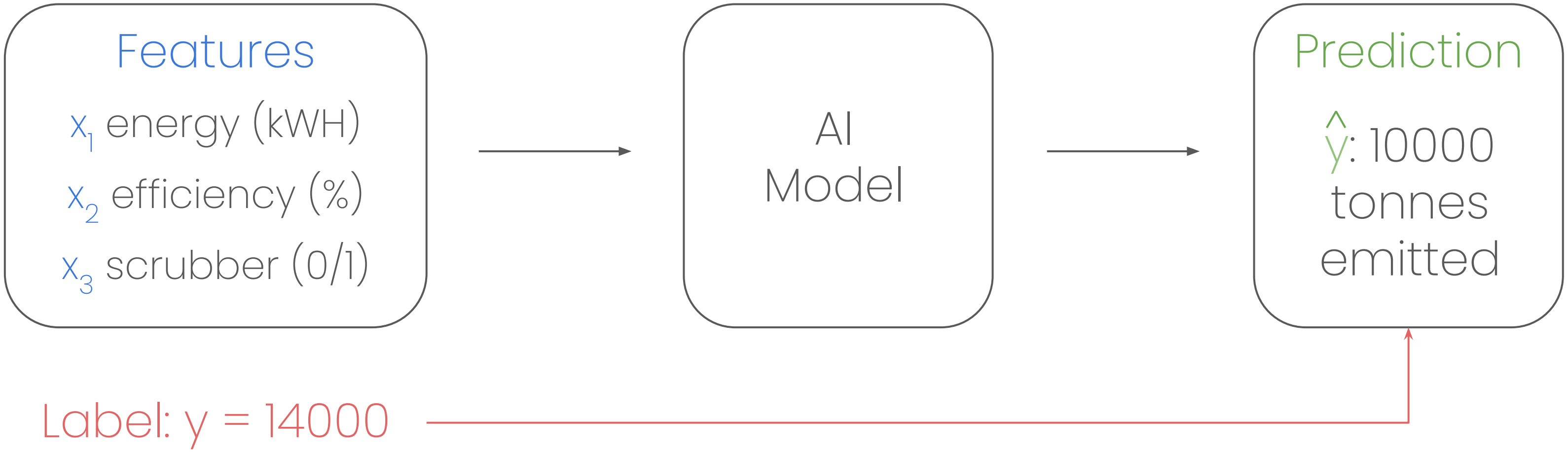
Supervised ML

AI Model

Prediction: Lynx

Label: Cat

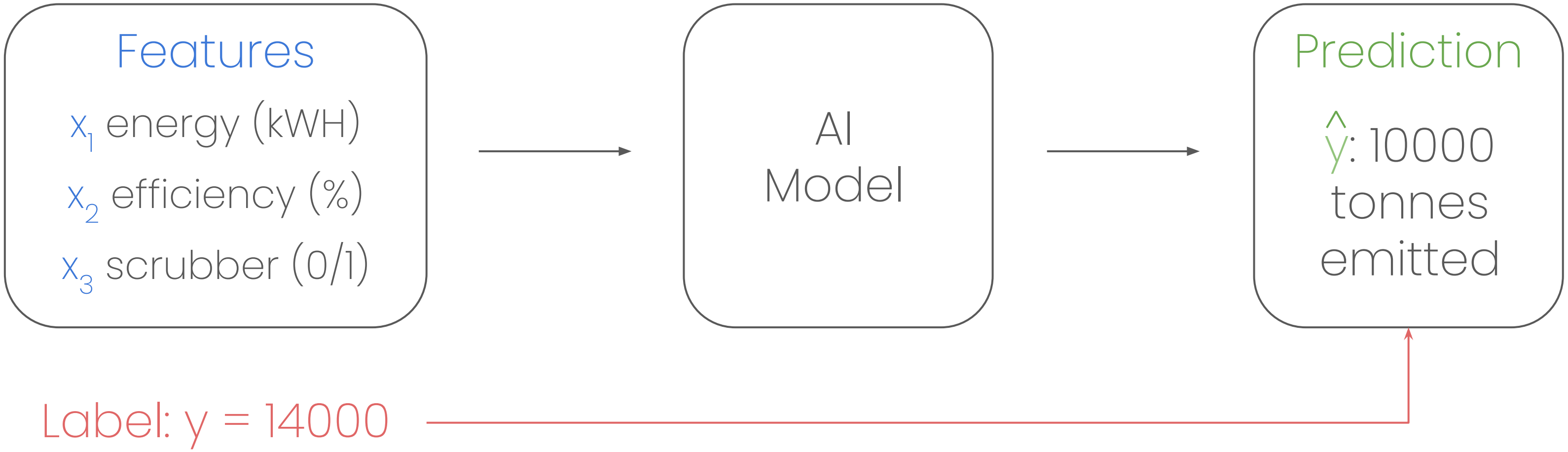
Ex: Power Plant CO_2 Emissions



3 Parts of Supervised ML

(Predictions, Error, Optimisation)

Creating Predictions



Creating Predictions

AI Model

Features

x_1 energy (kWH)

x_2 efficiency (%)

x_3 scrubber (0/1)

Prediction

$\hat{y}$: CO_2 emitted (tonnes)

Problem solved?

Linear Regression

$$\hat{y} = 2x_1 + 3x_2 + 4x_3$$

Features

x_1 energy (kWH)

x_2 efficiency (%)

x_3 scrubber (0/1)

Prediction

$\hat{y}$: CO_2 emitted (tonnes)

Nope: how to pick parameters?

Linear Regression

Features

x_1 energy (kWH)

x_2 efficiency (%)

x_3 scrubber (0/1)

$$\hat{y} = 2x_1 + 3x_2 + 4x_3$$

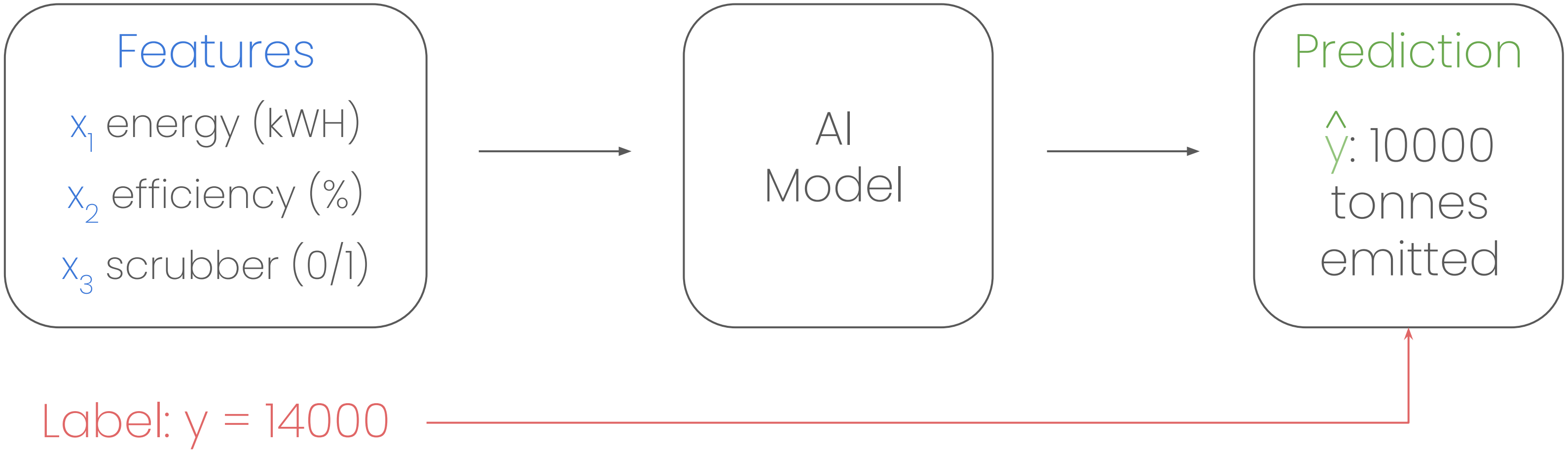
Prediction

$\hat{y}$: CO_2 emitted (tonnes)

3 Parts of Supervised ML

(Predictions, Error, Optimisation)

How to measure error?



Idea 1: Difference

$$y - \hat{y}$$

$$14000 - 10000$$

$$4000$$

Idea 2: Total Difference

$$\left(y^{(1)} - \hat{y}^{(1)}\right) + \left(y^{(2)} - \hat{y}^{(2)}\right)$$

$$(14000 - 10000) + (8000 - 12000)$$

$$0$$ 🎉

Idea 2: Total Squared Difference

$$(y^{(1)} - \hat{y}^{(1)})^2 + (y^{(2)} - \hat{y}^{(2)})^2$$

$$(14000 - 10000)^2 + (8000 - 12000)^2$$

32,000,000 😱

Idea 3: Mean Squared Difference

$$[(y^{(1)} - \hat{y}^{(1)})^2 + (y^{(2)} - \hat{y}^{(2)})^2 + \ldots + + (y^{(N)} - \hat{y}^{(N)})^2]/N$$

$$MSE = \frac{1}{N}\sum_{k=1}^{N}(y^{(k)} - \hat{y}^{(k)})^2$$

3 Parts of Supervised ML

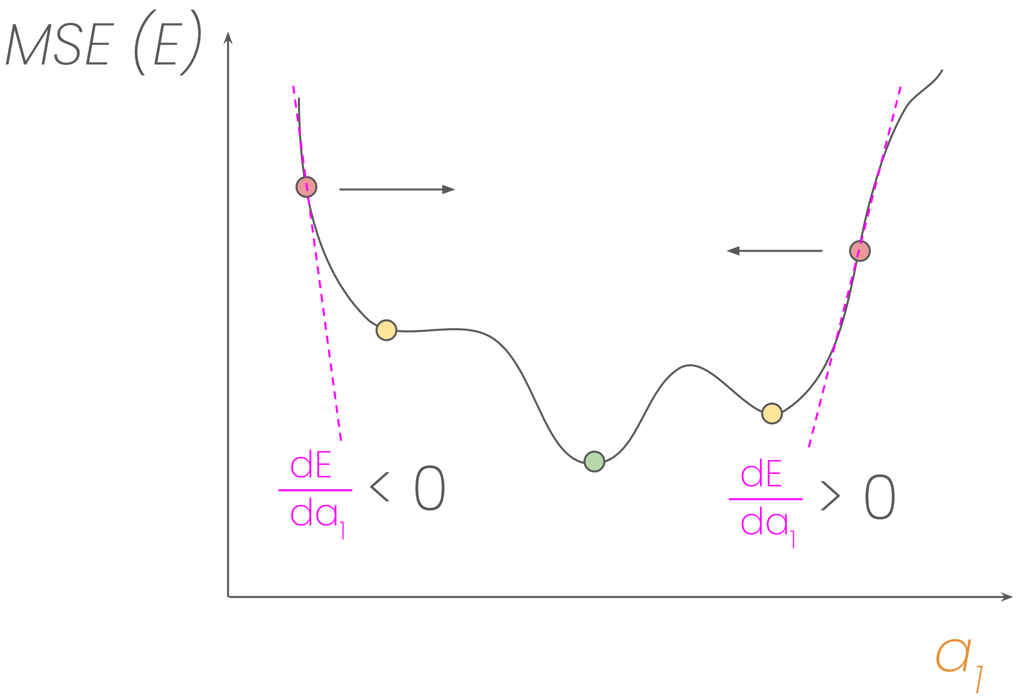
(Predictions, Error, Optimisation)

Gradient Descent

$$\hat{y} = a_1 x_1$$

$$MSE = \frac{1}{N}\sum_{k=1}^{N}\left(y^{(k)} - \hat{y}^{(k)}\right)^2$$

$$a_1 = a_1 - \frac{dE}{da_1}$$

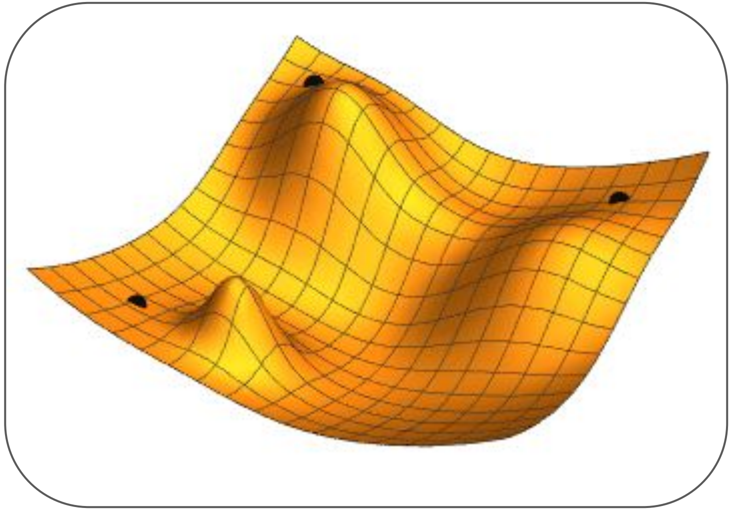


Gradient Descent

$$\hat{y} = a_1 x_1 + a_2 x_2 + \ldots$$

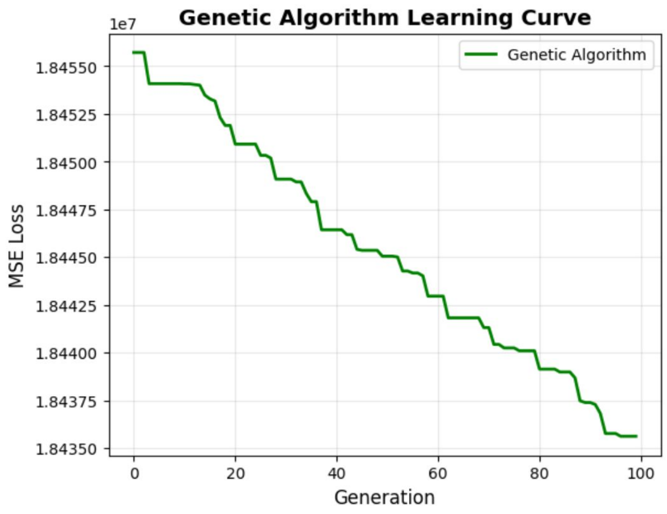
$$MSE = \frac{1}{N}\sum_{k=1}^{N}\left(y^{(k)} - \hat{y}^{(k)}\right)^2$$

$$\vec{a} = \vec{a} - \alpha \nabla E(a)$$



Genetic Algorithm (Smart Trial & Error)

1. Make 100 sets of random parameters (a_1, a_2, a_3, ...)
2. Check error for each set
3. Worst 5 sets replaced with new random sets.
4. Other sets get more 'mutations' with higher error. Ex: $\vec{a} = \vec{a} + E \cdot rand$.



🔗 Demo 🔗

3 Parts of Supervised ML

(Predictions, Error, Optimisation)

(Except More Complicated)

Logistic Regression

Linear Regression

$$\hat{y} = 2x_1 + 3x_2 + 4x_3$$

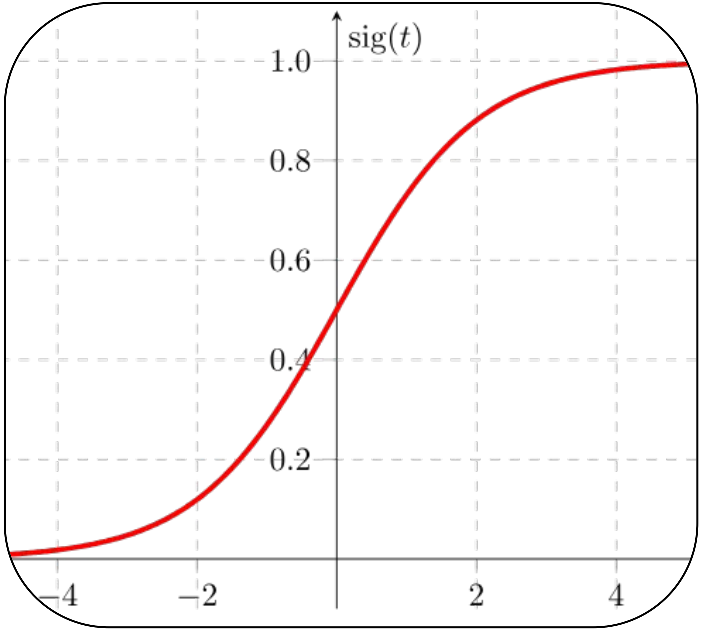
Logistic Regression

$$z = 2x_1 + 3x_2 + 4x_3$$

$$\hat{y} = \sigma(z)$$

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

Logistic Regression (probabilities)



Logistic Regression

$$z = 2x_1 + 3x_2 + 4x_3$$

$$\hat{y} = \sigma(z)$$

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

Sidenote: Binary Cross Entropy

MSE
(Big Numbers)

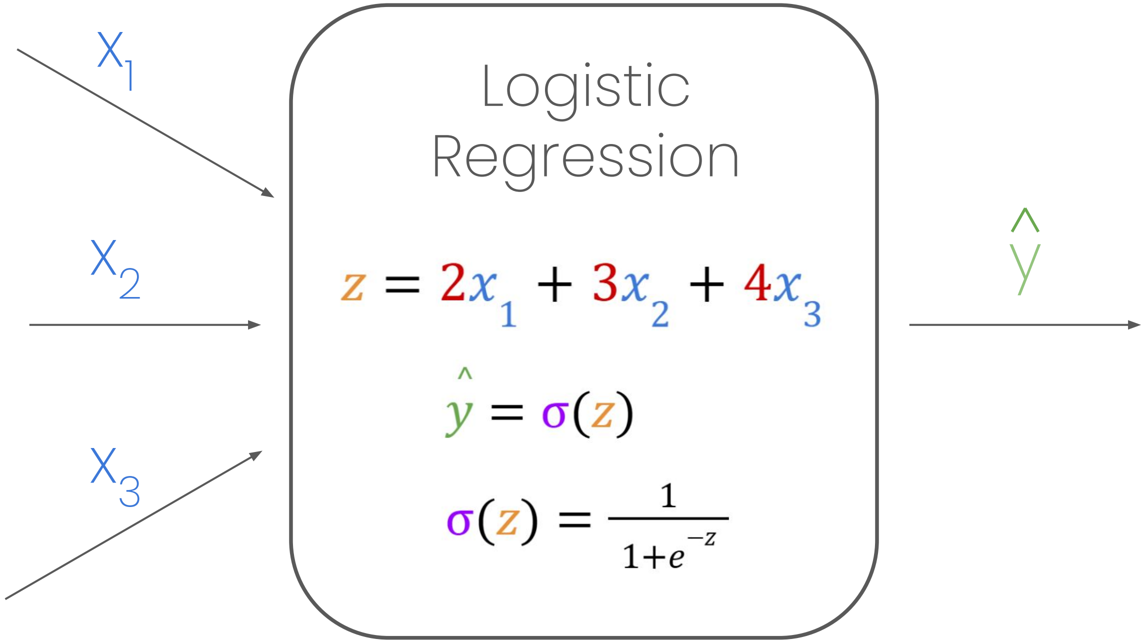
$$MSE = \frac{1}{N}\sum_{k=1}^{N}(y^{(k)} - \hat{y}^{(k)})^2$$

BCE
(Probabilities)

$$BCE = -\frac{1}{N}\sum_{k=1}^{N}[BCE_k]$$

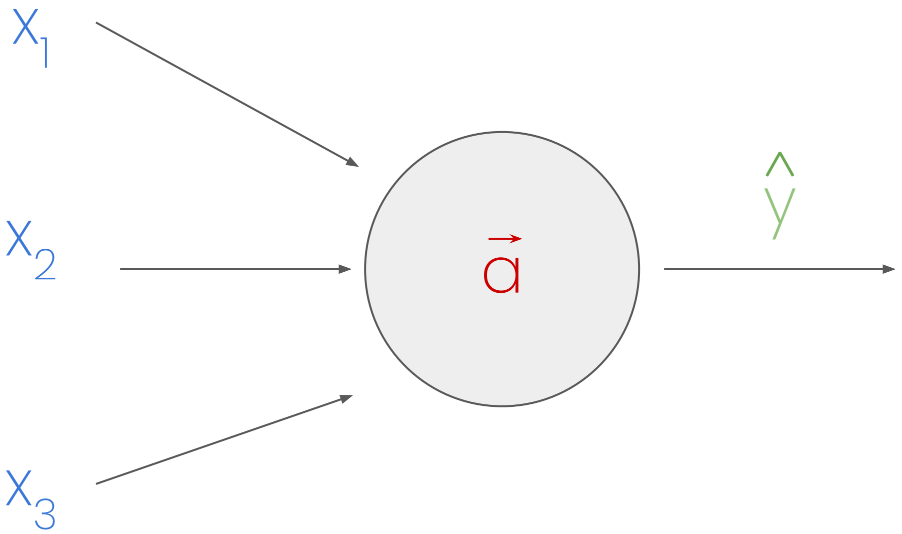
$$BCE_k = y^{(k)}\log(\hat{y}^{(k)}) + (1 - y^{(k)})\log(1 - \hat{y}^{(k)})$$

Perceptron

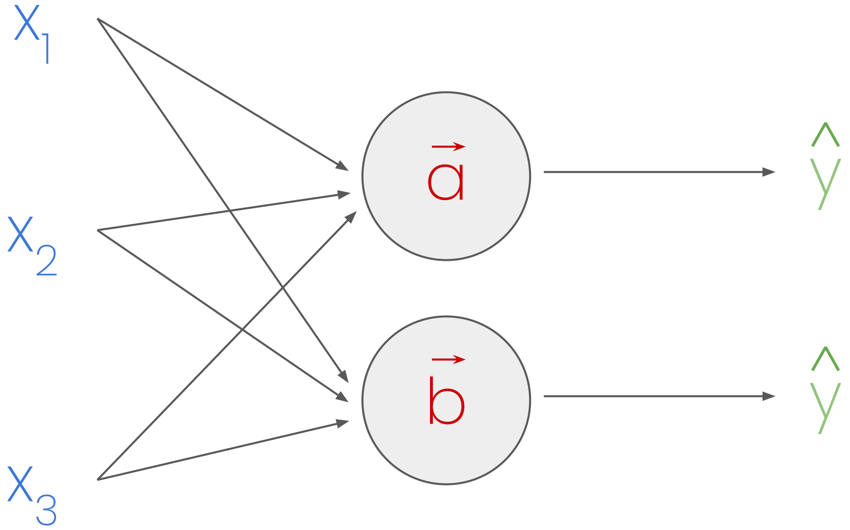


X_1
X_2
X_3
Logistic Regression
$z = 2x_1 + 3x_2 + 4x_3$
$\hat{y} = \sigma(z)$
$\sigma(z) = \frac{1}{1+e^{-z}}$
$\hat{y}$

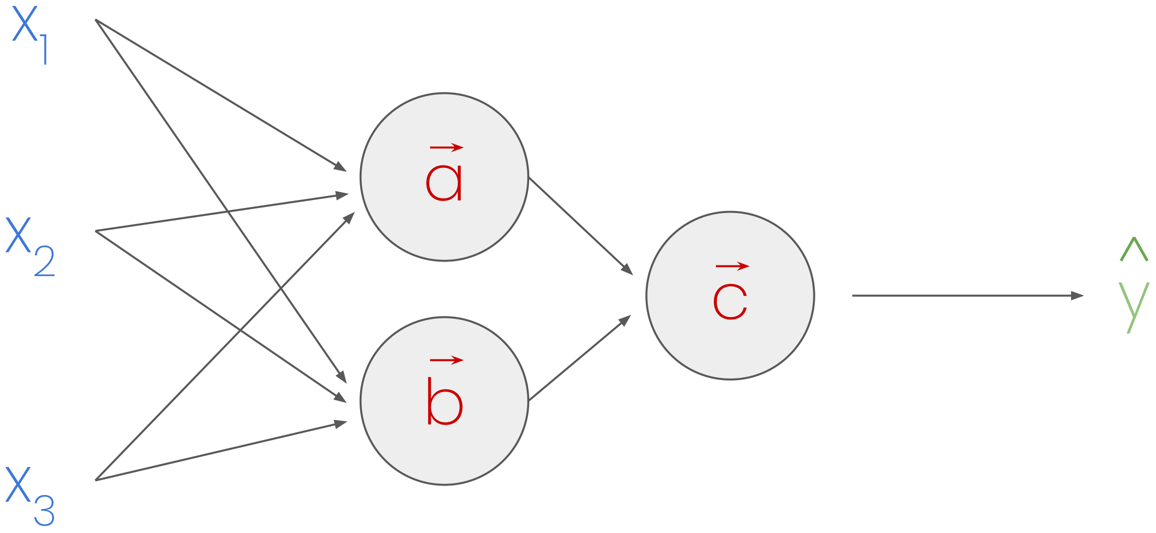
Multi-layer Perceptron



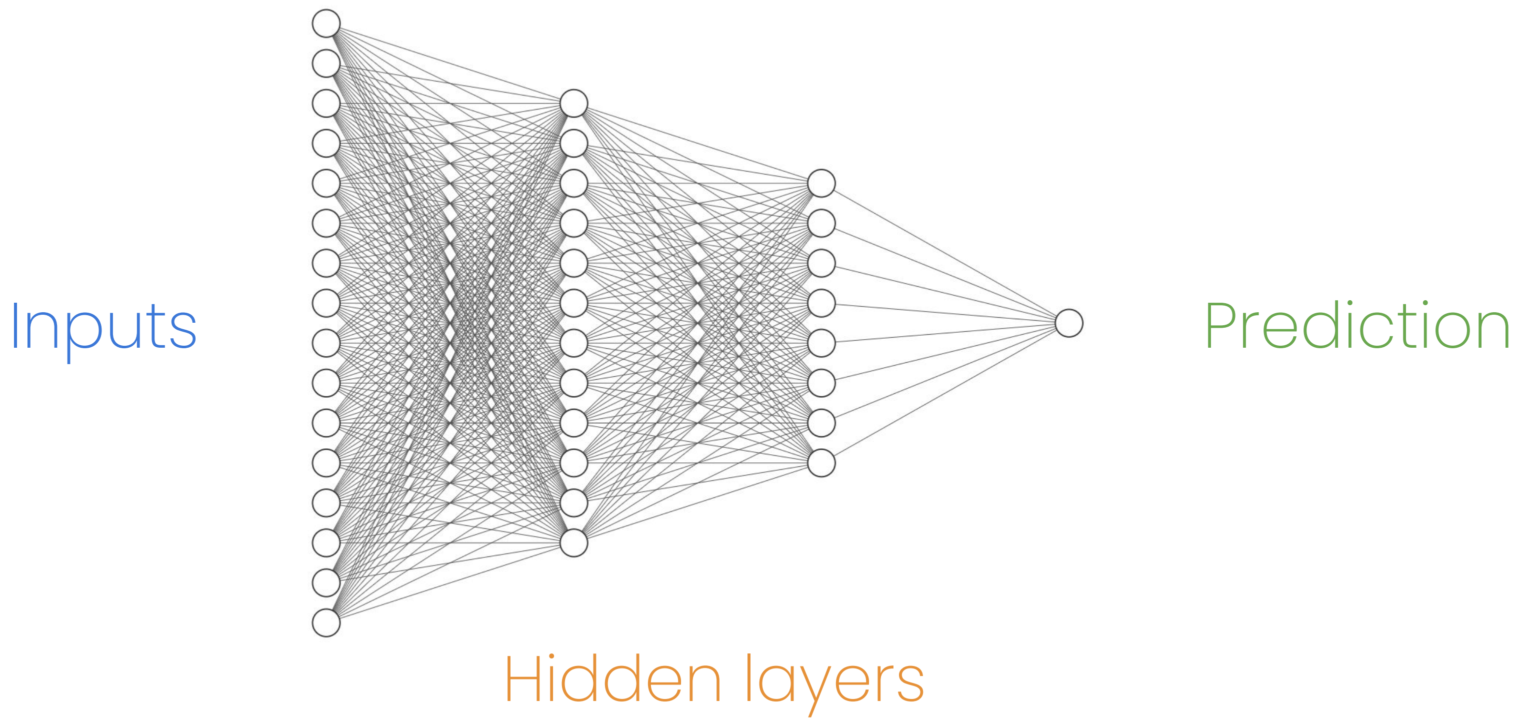
Multi-layer Perceptron



Multi-layer Perceptron



Neural Network



Activation Functions

Replace sigmoid as needed

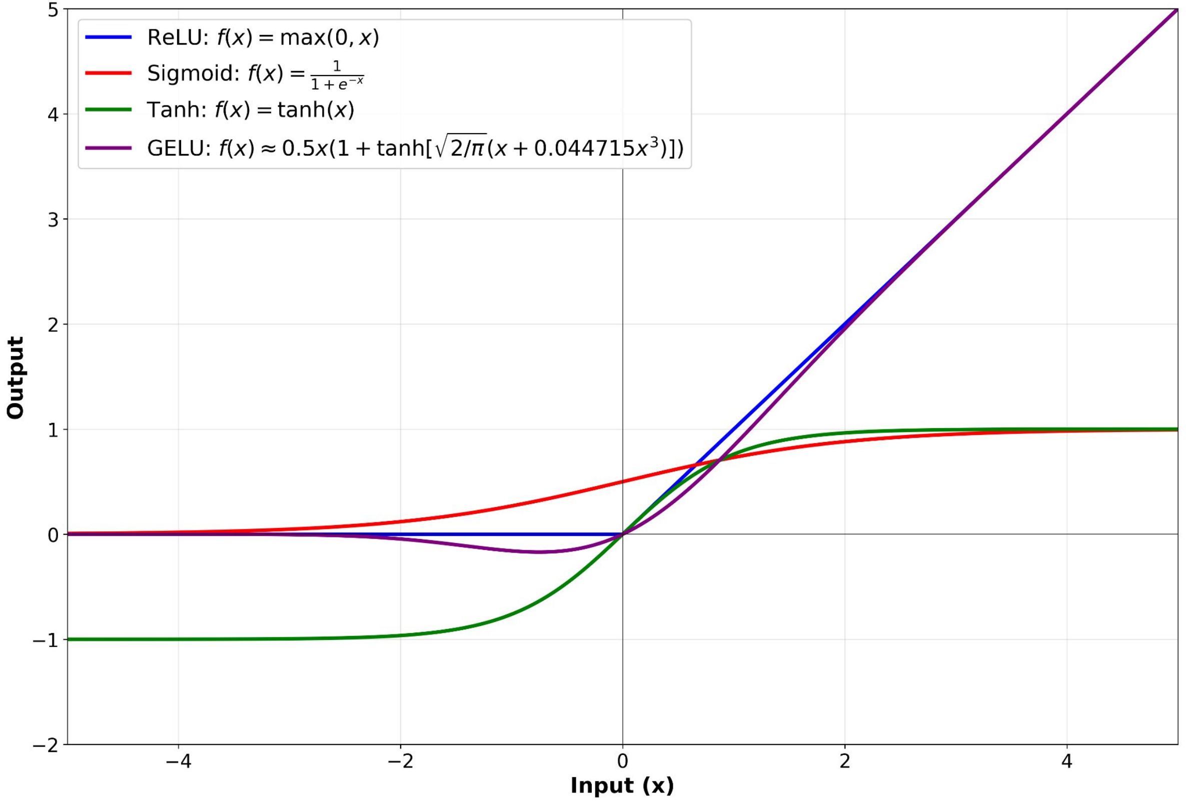
Neuron

$$z = 2x_1 + 3x_2 + 4x_3$$

$$\hat{y} = \sigma(z)$$

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

Activation Functions



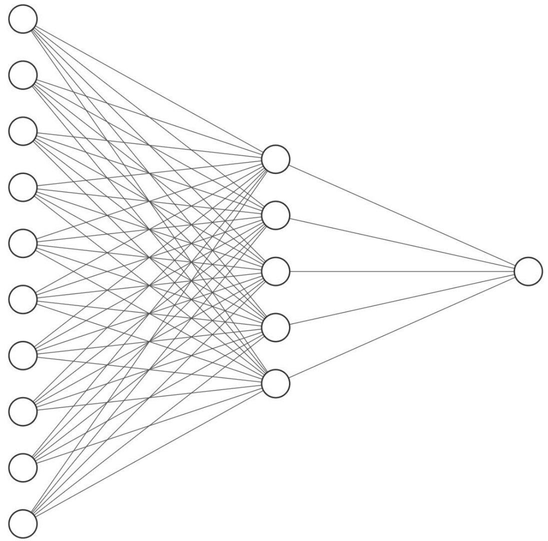
So just add fancy equations, no worries?

Gradient Descent (Revisited)

$$\hat{y} = a_1 x_1 + a_2 x_2 + \ldots$$

$$MSE = \frac{1}{N} \sum_{k=1}^{N} (y^{(k)} - \hat{y}^{(k)})^2$$

$$\vec{a} = \vec{a} - \alpha \nabla E(a)$$



Gradient Descent (New Equations)

$$\vec{z}_1 = W_1 \vec{x}_1 + \vec{b}_1$$ - $\vec{x}_1 \in \mathbb{R}^{10}$, $W_1 \in \mathbb{R}^{5\times10}$, $\vec{b}_1 \in \mathbb{R}^{5}$

$$\vec{a}_1 = \sigma(\vec{z}_1)$$ - $\vec{a}_1, \vec{z}_1 \in \mathbb{R}^{5}$

$$z_2 = W_2 \vec{a}_1 + \vec{b}_2$$ - $\vec{a}_1, \vec{b}_2 \in \mathbb{R}^{5}$, $W_2 \in \mathbb{R}^{1\times5}$

$$\hat{y} = \sigma(z_2)$$ - $\hat{y}, z_2 \in \mathbb{R}$

$$MSE = \frac{1}{N}\sum_{k=1}^{N}\left(y^{(k)} - \hat{y}^{(k)}\right)^2$$

Gradient Descent (New Equations)

$\vec{z}_1 = W_1\vec{x}_1 + \vec{b}_1$ - $\vec{x}_1 \in \mathbb{R}^{10}$, $W_1 \in \mathbb{R}^{5\times10}$, $\vec{b}_1 \in \mathbb{R}^5$

$\vec{a}_1 = \sigma(\vec{z}_1)$ - $\vec{a}_1, \vec{z}_1 \in \mathbb{R}^5$

$z_2 = W_2\vec{a}_1 + \vec{b}_2$ - $\vec{a}_1, \vec{b}_2 \in \mathbb{R}^5$, $W_2 \in \mathbb{R}^{1\times5}$

$\hat{y} = \sigma(z_2)$ - $\hat{y}, z_2 \in \mathbb{R}$

$$MSE = \frac{1}{N}\sum_{k=1}^{N}(y^{(k)} - \hat{y}^{(k)})^2$$

Forward Pass

$$W_1 = W_1 - \alpha \nabla E(W_1)$$

$$W_1 = W_1 - \alpha \nabla E(\hat{y}) \cdot \nabla \hat{y}(W_1)$$

$$W_1 = W_1 - \alpha \nabla E(\hat{y}) \cdot \boldsymbol{\sigma'}(z_2) \cdot \nabla z_2(W_1)$$

$$W_1 = W_1 - \alpha \hat{\nabla} E(y) \cdot \boldsymbol{\sigma'}(z_2) \cdot \vec{\nabla} z_2(\vec{a}_1) \cdot \nabla a_1(W_1)$$

...

Backpropagation

~~Gradient Descent~~ Chain Rule Round ∞

$\vec{z}_1 = W_1\vec{x}_1 + \vec{b}_1$ - $\vec{x}_1 \in \mathbb{R}^{10}$, $W_1 \in \mathbb{R}^{5\times10}$, $\vec{b}_1 \in \mathbb{R}^{5}$

$\vec{a}_1 = \sigma(\vec{z}_1)$ - $\vec{a}_1, \vec{z}_1 \in \mathbb{R}^{5}$

$z_2 = W_2\vec{a}_1 + \vec{b}_2$ - $\vec{a}_1, \vec{b}_2 \in \mathbb{R}^{5}$, $W_2 \in \mathbb{R}^{1\times5}$

$\hat{y} = \sigma(z_2)$ - $\hat{y}, z_2 \in \mathbb{R}$

$$MSE = \frac{1}{N}\sum_{k=1}^{N}\left(y^{(k)} - \hat{y}^{(k)}\right)^2$$

Forward Pass

$$W_1 = W_1 - \alpha \nabla E(W_1)$$

$$W_1 = W_1 - \alpha \nabla E(\hat{y}) \cdot \nabla \hat{y}(W_1)$$

$$W_1 = W_1 - \alpha \nabla E(\hat{y}) \cdot \boldsymbol{\sigma'}(z_2) \cdot \nabla z_2(W_1)$$

$$W_1 = W_1 - \alpha \hat{\nabla} E(y) \cdot \boldsymbol{\sigma'}(z_2) \cdot \vec{\nabla} z_2(\vec{a}_1) \cdot \nabla a_1(W_1)$$

...

Backpropagation

xkcd #1838


THIS IS YOUR MACHINE LEARNING SYSTEM?
YUP! YOU POUR THE DATA INTO THIS BIG PILE OF LINEAR ALGEBRA, THEN COLLECT THE ANSWERS ON THE OTHER SIDE.
WHAT IF THE ANSWERS ARE WRONG?
JUST STIR THE PILE UNTIL THEY START LOOKING RIGHT.
DATA
ANSWERS

🔗 Demo 2 🔗